

COMPUTER SCIENCE TRIPOS Part IB – 2020 – Paper 4

9 Semantics of Programming Languages (nk480)

Subtyping

Many languages (like C and Java) support *coercions*, in which values of one datatype (e.g., machine integers) can be used where values of another datatype (e.g., floating point numbers) are expected, by having the compiler silently insert code to convert from one type to another. Suppose we have a language with the following grammar of types:

$$T ::= \text{int} \mid \text{bool} \mid \text{string} \mid T \times T' \mid T \rightarrow T'$$

Suppose we then define a subtyping relation as follows:

$$\begin{array}{c} \frac{}{T \leq T} \qquad \frac{T \leq T' \quad T' \leq T''}{T \leq T''} \\[10pt] \frac{T_1 \leq T'_1 \quad T_2 \leq T'_2}{T_1 \times T_2 \leq T'_1 \times T'_2} \quad \frac{T'_1 \leq T_1 \quad T_2 \leq T'_2}{T_1 \rightarrow T_2 \leq T'_1 \rightarrow T'_2} \\[10pt] \frac{}{\text{bool} \leq \text{string}} \qquad \frac{}{\text{int} \leq \text{string}} \\[10pt] \frac{}{\text{bool} \leq \text{int}} \end{array}$$

- (a) Assuming the existence of functions `bool_to_string`, `int_to_string`, and `bool_to_int`, adapt the relation above to define a new relation $T \leq T' \rightsquigarrow e$, where e is a *coercion*, a closed function of type $T \rightarrow T'$. (You may use ML or lambda-calculus notation to define the coercions e .) [10 marks]
- (b) Explain what this relation could be used for in a language implementation. [2 marks]
- (c) Give definitions of `bool_to_string` and `bool_to_int`, and then use the relation you defined to give two subtyping derivations $\text{bool} \leq \text{string} \rightsquigarrow e_1$ and $\text{bool} \leq \text{string} \rightsquigarrow e_2$ such that e_1 and e_2 have different behaviour. [5 marks]
- (d) What problem would this lead to in a language implementation? [3 marks]

Answer:

— *Solution notes* —

(a) Here is the coercion relation:

$$\begin{array}{c}
 \frac{}{T \leq T \rightsquigarrow id_T} \qquad \frac{T \leq T' \rightsquigarrow f \quad T' \leq T'' \rightsquigarrow g}{T \leq T'' \rightsquigarrow g \circ f} \\
 \\
 \frac{T_1 \leq T'_1 \rightsquigarrow f \quad T_2 \leq T'_2 \rightsquigarrow g}{T_1 \times T_2 \leq T'_1 \times T'_2 \rightsquigarrow \lambda(x, y). (f\ x, g\ y)} \quad \frac{T'_1 \leq T_1 \rightsquigarrow f \quad T_2 \leq T'_2 \rightsquigarrow h}{T_1 \rightarrow T_2 \leq T'_1 \rightarrow T'_2 \rightsquigarrow \lambda g. h \circ g \circ f} \\
 \\
 \frac{}{bool \leq string \rightsquigarrow bool_to_string} \qquad \frac{}{int \leq string \rightsquigarrow int_to_string} \\
 \\
 \frac{}{bool \leq int \rightsquigarrow bool_to_int}
 \end{array}$$

(b) This relation lifts the base-type coercions to the coercion for arbitrary subtyping, and a compiler can insert the e terms to coerce arbitrary values as needed.

(c) Here are two definitions:

$$\begin{array}{ll}
 bool_to_int(false) & = 0 \\
 bool_to_int(true) & = 1 \\
 \\
 bool_to_string(false) & = \text{“false”} \\
 bool_to_string(true) & = \text{“string”}
 \end{array}$$

Now, we can produce the following two derivation trees:

$$\begin{array}{c}
 \frac{}{bool \leq string \rightsquigarrow bool_to_string} \\
 \\
 \frac{bool \leq int \rightsquigarrow bool_to_int \quad int \leq string \rightsquigarrow int_to_string}{bool \leq string \rightsquigarrow int_to_string \circ bool_to_int}
 \end{array}$$

(d) The first coercion can coerce `true` to “true”, and the second can coerce it to “0”. If both coercions are available, it is unspecified which one the language may choose.
